Advanced CSS and Sass: Flexbox, Grid, Animations and More!

Section 1: Welcome, Welcome, Welcome!

Welcome to the Most Advanced CSS Course Ever!

1. 7 sections - 20+ hours
2. Sections:
    1. Natours Project - Setup and First Steps (Part 1)
        1. Landing page
        2. New CSS properties and techniques
        3. Animations
        4. Advanced responsive design
        5. CSS architecture
        6. ...
    2. How CSS Works: A Look Behind the Scenes
        1. CSS architecture
    3. Introduction to Sass and NPM
        1. Tools to write and compile Sass
    4. Natours Project - Using Advanced CSS and Sass (Part 2)
        1. CSS properties and features
    5. Natours Project - Advanced Responsive Design (Part 3)
    6. Trillo Project - Master Flexbox!
    7. A Quick Introduction to CSS Grid Layouts
        1. Fundamentals
    8. Nexter Project - Master CSS Grid Layouts!

READ BEFORE YOU START!

1. Download starter files, final project code, and course slides
    1. Course material and instructions on GitHub
2. Discord community
3. Online tools and resources

Setting up Our Tools

1. VS Code
    1. Auto-complete
    2. Built-in git support
    3. Extensions
        1. Very smooth
2. Google Chrome

Section 2: Natours Project - Setup and First Steps (Part 1)

Section Intro

1. HTML and advanced CSS
2. Set up project
3. Exciting CSS features

Project Overview

1. Project
    1. Website - landing page
        1. Tours in the nature
    2. Animations
    3. Features
    4. Animated cards
    5. Popup (pure CSS and not JS)
    6. Background video
    7. Modern look and feel
    8. Placeholder slides down
    9. Custom radio buttons
    10. Navigation
        1. Animation
2. Open Natures project in VS Code: https://github.com/jonasschmedtmann/advanced-css-course

Building the Header - Part 1

1. What you will learn in this lecture

    1. The best way to perform a basic reset using the universal selector
    2. How to set project-wide font definitions
    3. How to clip parts of elements using `clip-path` (modern CSS property)

2. Universal selector:

    1. `*` - selects each and every element in the page

    2. Reset:

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

1. Browsers, by default, apply some margin and padding on some elements (main heading say)
2. `border-box` - change box model so that borders and paddings are no longer added to border width or border height specified

3. Properties related to font are inherited

```
body {
    font-family: "Lato", sans-serif; // inherited by all child elements in the body
    font-weight: 400;
    font-size: 16px;
    line-height: 1.7; // 1.7 times bigger than the predefined line-height
    color: #777;
}
```

3. https://bennettfeely.com/clippy/

4. Code: style.css

```
/*
COLORS:

Light green: #7ed56f
Medium green: #55c57a
Dark green: #28b485

*/

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Lato", sans-serif;
  font-weight: 400;
  font-size: 16px;
  line-height: 1.7;
  color: #777;
  padding: 30px; /* Not inherited */
}

.header {
  height: 95vh; /* 95% of viewport height */
  background-image: linear-gradient(
      to right bottom,
      rgba(126, 213, 111, 0.8),
      rgba(40, 180, 133, 0.8)
    ),
    url("../img/hero.jpg");
  background-size: cover; /* tries to fit image inside the box */
  background-position: top; /* top of the image stays on top */

  clip-path: polygon(0 0, 100% 0, 100% 75vh, 0 100%);
  /* clip-path: polygon(50% 0, 100% 100%, 0 100%); */
}
```

Building the Header - Part 2

1. What we will learn:

    1. The easiest way to center anything with the `transform`, `top`, and `left` properties

2. Code: style.css

```
.logo-box {
  position: absolute;
  top: 40px;
  left: 40px;
}

.logo {
  height: 35px;
}

.text-box {
  position: absolute;
  top: 50%; /* In relation to the parent element */
  left: 50%;
  transform: translate(
    -40%,
    -50%
  ); /* Relative to the element and not it's parent */
}

.heading-primary {
  color: #fff;
  text-transform: uppercase;
}

.heading-primary-main {
  display: block;
  font-size: 60px;
  font-weight: 400;
  letter-spacing: 35px;
}
```

```
.heading-primary-sub {
  display: block;
  font-size: 20px;
  font-weight: 700;
  letter-spacing: 17.4px;
}
```

3. Code: index.html

```
<body>
    <header class="header">
        <div class="logo-box">
            <img src="img/logo-white.png" alt="Logo" class="logo">
        </div>

        <div class="text-box">
            <h1 class="heading-primary"> <!-- Important for Google -->
                <span class="heading-primary-main">Outdoors
                </span>
                <span class="heading-primary-sub">is where life happens</span>
            </h1>
        </div>
    </header>
</body>
```

Constructing Cool CSS Animations

1. What you will learn in this
    1. How to contstruct CSS animations using `@keyframes` and the `animation` property
2. Browsers are optimized for
    1. Opacity
    2. Transform
3. ctrl+d (select multiple instances)

Building a Complex Animated Button - Part 1

1. What you will learn in the lecture
    1. What pseudo-elemets and pseudo-classes are;
    2. How and why to use the `::after` pseudo-element
    3. How to construct a creative hover animation effect using the `transition` property
2. Pseudo-class - a special state of an element
3. Code: style.css

```
.btn:link,
.btn:visited {
  text-transform: uppercase;
  text-decoration: none;
  padding: 15px 40px;
  display: inline-block;
  border-radius: 100px;
  transition: all 0.2s;
}

.btn:hover {
  transform: translateY(-3px);
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2); /* 0px - x axis, 10px down - y axis, 20px - blur  */
}

.btn:active {
  transform: translateY(-1px); /* in relation to original state */
  box-shadow: 0 5px 10px rgba(0, 0, 0, 0.2);
}

.btn-white {
  background-color: #fff;
  color: #777;
}
```

4. Code: index.html

```
<a href="#" class="btn btn-white">Discover our tours</a>
```

5. Pseudo elements: Allow us to style certain parts of elements
    1. It is a virtual element after the element under selection which can be styled

Building a Complex Animated Button - Part 2

1. Code: style.css

```
@keyframes moveInBotton {
  0% {
    opacity: 0;
    transform: translateY(30px);
  }

  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

.btn:link,
```

```
.btn:visited {
  text-transform: uppercase;
  text-decoration: none;
  padding: 15px 40px;
  display: inline-block;
  border-radius: 100px;
  transition: all 0.2s;
  position: relative;
}

.btn:hover {
  transform: translateY(-3px);
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2); /* 0px - x axis, 10px down - y axis, 20px - blur  */
}

.btn:active {
  transform: translateY(-1px); /* in relation to original state */
  box-shadow: 0 5px 10px rgba(0, 0, 0, 0.2);
}

.btn-white {
  background-color: #fff;
  color: #777;
}

/* Acts like a child of `btn` */
.btn::after {
  content: "";
  display: inline-block;
  height: 100%; /* same height as parent: `btn` */
  width: 100%;
  border-radius: 100px;
  position: absolute;
  top: 0;
  left: 0;
  z-index: -1; /* Behind the button */
  transition: all 0.4s;
}

.btn-white::after {
  background-color: #fff;
}

/* On hover of button apply style to pseudo element ::after */
.btn:hover::after {
  transform: scaleX(1.5) scaleY(1.6);
  opacity: 0;
}

.btn-animated {
  animation: moveInBotton 0.5s ease-out 0.75s;
  animation-fill-mode: backwards; /* It automatically applies the style of 0% before the animation starts */
}
```

Section 3: How CSS Works: A Look Behind the Scenes

Section Intro

1. Important and advanced concepts

Three Pillars of Writing Good HTML and CSS (Never Forget Them!)

1. Three pillars to write good HTML and CSS... and build good websites
    1. Responsive design
    2. Maintainable and scalable code
    3. Web performance

Responsive Design

1. One website that works for all screen sizes on all devices
    1. Concepts:
        1. Fluid layouts
        2. Media queries
        3. Responsive images
        4. Correct units
            1. Font sizes
            2. Element dimensions
        5. Desktop-first vs mobile-first

Maintainable and Scalable Code

1. Topics - For professional Web development
    1. Clean
    2. Easy-to-understand
    3. Growth
    4. Reusable
    5. How to organize files (part of CSS architecture)
    6. How to name classes (part of CSS architecture)
    7. How to structure HTML (part of CSS architecture)

Web Performance

1. Performance: Faster and smaller (user needs to download less data)

2. Topics:
    1. Less HTTP requests
    2. Less code
    3. Compress code
    4. Use a CSS preprocessor
    5. Less images
        1. The biggest part of the size
            1. Use only necessary images
    6. Compress images
        1. They use less bandwidth

How CSS Works Behind the Scenes: An Overview

1. To become a better CSS developer

What Happens to CSS When We Load Up a Web Page?

1. Flow:
    1. Load HTML
    2. Parse HTML (decodes code line by line)
        1. Browser builds Document Object Model (DOM)
            1. Describes web document as a family tree (decoded HTML code)
    3. Browser loads CSS mentioned in the header
    4. Parse CSS (more complex)
        1. **Resolve conflicting CSS declarations (cascade)**
        2. **Process final CSS values**
            1. % to px
    5. CSS Object Model (CSSOM)
        1. Final CSS is stored in a tree-like structure
    6. DOM + CSSOM => Render Tree
    7. Website rendering: the visual formatting model
        1. The visual formatting model: Algorithm calculates and uses many things (box model, floats, positioning)
    8. Final rendered website

How CSS is Parsed, Part 1: The Cascade and Specificity

Quick Review: CSS Terminology

1. A CSS Rule

```
.my-class {
    color: blue;
    text-align: center;
    font-size: 20px;
}
```

1. `.my-class` - selector
    1. To select one or more HTML elements
2. `{...}` - declaration block
    1. Actual style
3. `color: blue` - declaration
    1. `color` - property
    2. `blue` - declared value

The Cascade (The "C" in CSS)

1. Cascade: It is the process of combining different stylesheets and resolving conflicts between different CSS rules and declarations, when more than one rule applies to a certain element.
    1. Sources:
        1. Author declarations
            1. Declarations in stylesheets
        2. User declarations
            1. Users changing style in browser
        3. Browser (user agent)
            1. Default styles applied by browser (if we don't style them)
                1. Set by browser
    2. Strategy to resolve conflict

```
importance (weight) > specificity > source order
```

        1. Cascading gives conflicting declarations different importances based on where they are declared
            1. User `!important` declarations (declarations marked with `!important` keyword)
            2. Author `!important` declarations
            3. Author declarations
            4. User declarations
            5. Default browser declarations (least important)
        2. Example: Author declarations

```
.button {
    font-size: 20px;
    color: white;
    background-color: blue !important; /* more important than the next one */
}

#nav .pull-right .button {
```

```
    background-color: green;
}
```

3. If conflicting rules exist without `!important` then they have the same importance
    1. Next Cascade compares specificities of the declarations
4. Specificities order:
    1. Inline styles (highest specificity)
    2. IDs
    3. Classes, pseudo-classes, attribute
    4. Elements, pseudo-elements [selectors] (least specificity)
5. Calculating specificity:

```
.button {
    font-size: 20px;
    color: white;
    background-color: blue;
}

nav#nav div.pull-right .button {
    background-color: green;
}

a {
    background-color: purple;
}

#nav a.button:hover {
    background-color: yellow;
}
```

    1. Specificity: `(inline, IDs, Classes, Elements)`
        1. For each one, we count the number of occurrences in the selector
        2. Example: (above)

```
(0, 0, 1, 0)
(0, 1, 2, 2) // (-, #nav, .pull-right & .button, nav & div)
(0, 0, 0, 1)
(0, 1, 2, 1) // .button & :hover
```

            1. We look from left to right (most specific to least specific)
                1. If a specificity is the highest, that wins
                    1. `(0, 1, 2, 2)` wins
                        1. It is the cascaded value
    2. If same specificity?
        1. The last declaration in the code will override all other declarations and will be applied

Cascade and Specificity: What you need to know

1. CSS declarations marked with `!important` have the highest priority
2. But, only use `!important` as a last resort. It's better to use correct specificities - **more maintainable code!**
    1. Can cause problems in the long run
3. Inline styles will always have priority over styles in external stylesheets
    1. Not a good practice
4. A selector that contains 1 ID is more specific than one with 1000 classes.
5. A selector that contains 1 class is more specific than one with 1000 elements.
6. The universal selector * has no specificity value (0, 0, 0, 0)
    1. All other selectors have presedence over it
7. Rely more on **specificity** than on the **order** of selectors
    1. If we re-arrange CSS code, we will not have surprises
        1. Easier to maintain
8. But, rely on order when using 3rd party stylesheets - always put your author stylesheet last
    1. We should be able to override the styles

Specificity in Practice

1. Codepen.io
    1. HTML

```
<nav id="nav">
  <div class="pull-right">
    <a class="button button-danger" href="link.html">Don't click here!</a>
  </div>
</nav>
```

    2. CSS:

```
body {
  padding: 50px;
}

.button {
  font-size: 20px;
  color: white;
  background-color: blue;
}

a {
  background-color: purple;
```

```
}

#nav div.pull-right a.button {
  background-color: orangered;
}

#nav div.pull-right a.button:hover { /* this class should have a higher specificity than the original one without :hover */
  background-color: green;
}

#nav a.button:hover {
  background-color: yellow; /* doesn't work because the specificity is lower */
}
```

1. Helps in debugging if something doesn't work

How CSS is Parsed, Part 2: Value Processing

1. How values are processed in parsing phase

How CSS Values are Processed

1. Each time we use a unit other than px (%, rem, ...), it will ultimately be converted to px.
    1. Helps us predict and control which property takes with value
2. Example:

```
<div class="section">
    <p class="amazing">CSS is absolutely amazing</p>
</div>

.section {
    font-size: 1.5rem;
    width: 280px;
    background-color: orangered;
}

p {
    width: 140px;
    background-color: green;
}

.amazing {
    width: 66%;
}
```

    1. 6 steps:
        1. Declared value (author declaration)
            1. width (paragraph):
                1. 140px
                2. 66% - more specific
            2. padding (paragraph):
            3. font-size (root):
            4. font-size (section):
            5. font-size (paragraph):
        2. Cascaded value (after the cascade)
            1. width (paragraph):
                2. 66% - more specific
            2. padding (paragraph):
            3. font-size (root):
            4. font-size (section):
            5. font-size (paragraph):
        3. Specified value (defaulting, if there is no cascaded value)
        4. Computed value (converting relative values to absolute - they can be inherited) - keywords such as `auto`, `orange`, `bolder`, ... are computed and replaced
            1. width (paragraph):
                2. 66% (nothing happens in this step because it is not a unit)
            2. padding (paragraph):
            3. font-size (root):
            4. font-size (section):
            5. font-size (paragraph):
        5. Used value (final calculations, based on layout)
            1. width (paragraph):
                2. 66% of 280px = 184.8px
            2. padding (paragraph):
            3. font-size (root):
            4. font-size (section):
            5. font-size (paragraph):
        6. Actual value (considered using browser and device restrictions)
            1. width (paragraph):
                2. 184.8px => 185px
            2. padding (paragraph):
            3. font-size (root):
            4. font-size (section):
            5. font-size (paragraph):
3. For each and every element in the page, each and every CSS property needs to have a value (even if not declared)
    1. padding (paragraph)
        1. Declared value (author declarations) -
        2. Cascaded value (after the cascade) -

3. Specified value (defaulting if there is no cascaded value)
    1. 0px (initial value) - already in absolute unit (if no padding is declared, we don't want any)
        1. Used if there is no cascaded value
            1. Inheritance also plays a role here
4. Computed value (converting relative vals to absolute)
5. Used value (final calculations, based on layout)
6. Actual value (browser and device restrictions)

4. Root: font-size:

    1. Declared value -
    2. Cascaded value (after the cascade) - 16px (Browser default - user agent declaration)
        1. No more calculation needed

5. Section: font-size:

    1. Declared value
        1. 1.5rem
    2. Cascaded value
        1. 1.5rem
    3. Specified value
        1. 1.5rem
    4. Computed value
        1. 24px (1.5rem x 16px)
    5. Used value
        1. 24px
    6. Actual value
        1. 24px

6. Paragraph: font-size

    1. Declared value
        1.
    2. Cascaded value
        1.
    3. Specified value
        1. 24px (inheritance)
            1. Some elements inherit the computed value of their parent
    4. Computed value
        1. 24px (1.5rem x 16px)
    5. Used value
        1. 24px
    6. Actual value
        1. 24px

How Units are Converted from Relative to Absolute (PX)

1. Example:

```
html, body {
    font-size: 16px;
    width: 80vw;
}

header {
    font-size: 150%;
    padding: 2em;
    margin-bottom: 10rem;
    height: 90vh;
    width: 1000px;
}

.header-child {
    font-size: 3em;
    padding: 10%;
}
```

    1. Example (x):
        1. % (fonts)
            1. 150%
        2. % (lengths) (reference is always parent element's width)
            1. 10%
        3. em (font) (font-based) (uses parent's font-size as reference)
            1. 3em
        4. em (lengths) (font-based) (uses parent's font-size as reference)
            1. 2em
        5. rem (font-based) (uses root's font-size as reference)
            1. 10rem
    2. How to convert to pixels
        1. % (fonts)
            1. x% * parent's computed font-size
                1. html/body
        2. % (lengths)
            1. x% * parent's computed width
                1. 10% * 1000px
        3. em (font)
            1. x * parent computed font-size
                1. 3 * 24px

4. em (lengths)
    1. x * current element computed font-size
        1. 2 * 24px
5. rem
    1. x * root computed font-size
        1. 10 * 16px

3. Result in pixels
    1. % (fonts)
        1. 24px
    2. % (lengths)
        1. 100px
    3. em (font)
        1. 72px
    4. em (lengths)
        1. 48px
    5. rem
        1. 160px

2. Why using font-size and ems and rems

    1. Enables building more responsive layouts
        1. Lengths can be changed by just changing font-size
            1. For flexibility

3. `vh`

    1. Example (x)
        1. 90vh
    2. How to convert to pixels
        1. x * 1% of viewport height
    3. Result in pixels
        1. 90% of the current viewport height (browser knows what the viewport height is)

4. `vw`

    1. Example (x)
        1. 80vw
    2. How to convert to pixels
        1. x * 1% of viewport width
    3. Result in pixels
        1. 80% of the current viewport width

CSS Value Processing: What you need to know

1. Each property has an initial value, used if nothing is declared (and if there is no inheritance - see next lecture)
2. Browsers specify a **root font-size** for each page (usually 16px)
3. Percentages and relative values are always converted to pixels
    1. For CSS engine to render the page in the screen
4. Percentages are measured relative to their parent's font-size, if used to specify font-size
5. Percentages are measured relative to their parent's width, if used to specify lengths
6. em are measured relative to their parent font-size, if used to specify font-size
7. em are measured relative to the current font-size, if used to specify lengths
8. rem are always measured relative to the document's root font-size
9. vh and vw are simply percentage measurements of the viewport's height and width

How CSS is Parsed, Part 3: Inheritance

Inheritance in CSS

1. Example: Let's analyse line-height on .child

```
.parent {
    font-size: 20px;
    line-height: 150%;
}

.child {
    font-size: 25px;
}
```

    1. Every CSS property must have a value
        1. Even if the author nor the browser specifies it
            1. (no cascaded value)
    2. Is there a cascaded value?
        1. If yes:
            1. Specified value = cascaded values
        2. If no:
            1. Is the property inherited? (specified to each property)
                1. Yes:
                    1. Some are inherited, and others are not
                    2. `line-height` property specification
                        1. It is inherited
                    3. Specified value = Computed value of parent element
                        1. It is not the % that is inherited, but the computed value
                2. No:
                    1. Secified value = Initial value (specific to each property)

Inheritance in CSS

1. Example:

```
.parent {
    font-size: 20px;
    line-height: 150%;
}

.child {
    font-size: 25px;
}
```

    1. Let's analyse `line-height` on `.child`

2. Steps:

    1. Every CSS property must have a value
    2. Is there a cascaded value?
        1. Yes
            1. Specified value = Cascaded value
        2. No
            1. Is the property inherited? (specific to each property)
                1. Yes
                    1. Specified value = Computed value of parent element
                        1. 30px (computed value)
                2. No
                    1. Specified value = Initial value (specific to each property)
                        1. Padding: 0px (intuitive)

Inheritance: What You Need to Know

1. Inheritance passes the value for some specific properties from parents to children - **more maintainable code**
    1. Advantage:
        1. Allows developer to write less code and code that is more maintainable
2. Poperties related to text are inherited:
    1. `font-family`
    2. `font-size`
    3. `color`
    4. etc.
3. Properties like margins and paddings are not inherited
    1. Impractical
        1. All children inheriting margin doesn't make sense
4. The computed value of a property is what gets inherited, not the declared value
5. Inheritance of a property only works if no one declares a value for that property
6. The `inherit` keyword forces inheritance on a certain property
7. The `initial` keyword resets a property to its initial value

Converting px to rem: An Effective Workflow

What You Will Learn in this Lecture

1. How and why to use `rem` units in our project
2. A great workflow for converting px to rem

Why?

1. We want an easy way to change measurements in a page with one simple setting
    1. Example: When we hit a breakpoint to display on a mobile device
        1. We want to decrease all measurements at the same time
            1. Instead of writing a lot of code to do that in media queries. One global setting can be changed. `font-size` say
2. Change font-size and px to rem

```
html {
    font-size: 10px;
}

/* ... */
border-radius: 10rem; /* 10 * 10px = 100px */
```

    1. Using `px` is also a bad practice
        1. Users cannot change the font-size in the browser (it is hardcoded)
            1. Solution: Change to `%`
                1. Example:

```
html {
  /* font-size: 10px; */ /* 1rem = 10px */
  font-size: 62.5%;
}
```

            2. `rem` not supported below IE 9 - irrelevant
3. Inheritance must be used instead of universal selector
4. Code: style.css

```
* *::after *::before {
  /* after and before don't get the style by default */
  margin: 0;
```

```
    padding: 0;
    /* box-sizing: border-box; */
    box-sizing: inherit; /* Inherits whatever we put inside `body` which is `box-sizing: border-box` */
  }

  body {
    font-family: "Lato", sans-serif;
    font-weight: 400;
    /* font-size: 16px; */
    line-height: 1.7;
    color: #777;
    /* padding: 30px; /* Not inherited */
    padding: 4rem;

    box-sizing: border-box; /* makes it easier to change in plugins. Better practicing */
  }
```

How CSS Renders a Website: The Visual Formatting Model

The Visual Formatting Model

1. **Definition**: It is an algorithm that calculates boxes and determines the layout of these boxes, for each element in the render tree, in order to determin the final layout of the page.
    1. Factors considered
        1. **Dimensions of boxes**: the box model
        2. **Box type**: inline, block, inline-block
        3. **Positioning scheme**: floats and positioning
        4. **Stacking contexts**
        5. Other elemnts in the render tree
            1. Siblings
            2. Parents
        6. Viewport size, dimensions of images, etc.

The Box Model

```
                    -
                  margin
                    -
                  padding
                    -
                    ^
                    |
margin | padding | <--- width ---> | padding | margin
                  height
                    |
                    v
                    -
                  padding
                    -
                  margin
                    -
```

```
1. Each and every element in a web-page can be seen as a box
2. Each box can have
    1. Width
    2. Height
    3. Padding (optional)
    4. Margin (optional)
    5. Border (optional)
3. **Content**
    1. text, images, etc. (we specify)
    2. We can define its `height` and `width` using the CSS properties.
4. **Padding**
    1. **Transparent area** around the content, inside of the box
    2. It is used to define whitespace inside of the box
    3. `padding` property can be used in CSS
5. **Border**
    1. Goes around the padding and the content
6. **Margin**
    1. Space between boxes (outside the box)
    2. Used to define space between two elements
7. **Fill area**
    1. Area that gets filled with background color or background image
        1. Includes **content area + padding area + border area**
```

The Box Model: Heights and Widths

1. **total width** = right border + right padding + specified width + left-padding + left border
2. **total height** = top border + top padding + specified height + bottom padding + bottom border
    1. Example: height = 0 + 20px + 100px + 20px + 0 = 140px
3. If we don't specify height and width, the visual formatting model will use content of the box to determine its size

The Box Model with Box-Sizing: Border-Box

1. width = left border + left padding + content width + right padding + right border
2. height = top border + top padding + content height + bottom padding + bottom border
3. **total width** = specified width
4. **total height** = specified height

Box Types: Inline, Block-Level, and Inline-Block

1. Block-Level boxes
    1. Elements formatted visually as blocks
    2. 100% of parent's width (as much space as possible)

3. Vertically, one after another
    1. Line breaks are created at the end
4. Box-model applies as shown
5. `display: block` (The below also produce block-level boxes)
    1. `display: flex`
    2. `display: list-item`
    3. `display: table`
6. Some elements' display property is set to `block` by default

2. Inline boxes
    1. Content is distributed in lines
        1. Only occupies the space its content needs
    2. Occupies only **content's space**
    3. No line-breaks
        1. They sit inside block level parent element
    4. No heights and widths
        1. They do not apply
    5. Paddings and margins only horizontal (left and right)
    6. `display: inline`
3. Inline-block boxes
    1. A mix of block and inline
    2. Occupies only content's space
    3. No line-breaks
    4. Box-model applies as shown
    5. `display: inline-block`

Positioning Schemes: Normal Flow, Absolute Positioning, And Floats

1. Normal flow
    1. Default position scheme
        1. Elements are layed out in the natural order of the code
    2. If **NOT** floated
    3. If **NOT** absolutely positioned
    4. Elements laid out according to their source order
    5. **Default**
        1. `position: relative` - it will still be normal flow
2. Floats
    1. **Element is removed from the normal flow**
        1. Shifted to left or right as far as possible until it touches the edge of its containing box or another float element
    2. Text and inline elements will wrap around the floated element
    3. The container will not adjust its height to the element
        1. Solution: Using clear fixes
    4. Types:
        1. `float: left`
        2. `float: right`
3. Absolute positioning
    1. **Element is removed from the normal flow**
    2. No impact on surrounding content or elements
        1. It can overlap other elements
    3. We use `top`, `bottom`, `left`, and `right` to offset the element from its relatively positioned container
        1. Stacking contexts
    4. Types:
        1. `position: absolute`
        2. `position: fixed`

Stacking Contexts

1. Determine in which order elements are rendered on the page

    1. A new stacking context can be constructed using different CSS properties

        1. Example: z-index

    2. Layers on the bottom of the stack are painted first and then subsequent layers on the top

    3. Example:

```
z-index: 3
position: relative

z-index: 2
position: absolute

z-index: 1
position: relative
```

        1. Stacking context with higher z-index appears on top

    4. Other properties that construct stacking contexts

        1. Opacity different from 1
        2. Transform
        3. Filter

    5. Sometimes the stacking order doesn't work as expected even after setting the z-index

CSS Architecture, Components and BEM

The Think - Build - Architect Mindset

1. We want code that is:
    1. Clean
    2. Modular
    3. Reusable
    4. Ready for growth (to add more features)
2. Architecture
    1. We need to take important decisions concerning HTML and CSS code right from the beginning
        1. We need good strategy
3. Strategy:

```
Think ---------------------------> Build ----------------------------> Architect
Think about the layout of your          Build your layout in HTML and           Construct a logical **architecture** for
webpage or web app before                       CSS with a consistent structure         your CSS with files and folders
writing code                                            for naming classes
```

Thinking About the Layout

1. Component-driven design
    1. **Modular building blocks** that make up interfaces
        1. Interface - collection of components held together by overall layout of the page
        2. Components should be **Re-usable** across a project, and between different projects
            1. We can build a library of components and re-use them across the project which will speed up development
        3. **Components** should be independent, allowing us to use them anywhere on the page
            1. Components should not depend on parent elements
        4. Similar to **Atomic design** philosopht
            1. Atoms -> Molecules -> Organisms -> Templates -> Pages
                1. Atoms - smallest units of the page
                    1. They combine to form molecules
                    2. Molecules compile to form organisms
                        1. Organisms - components

Building with Meaningful Class Names

1. Consistent strategy, structure for naming classes
    1. Strategies
        1. Object Oriented CSS
        2. SMAX?
        3. BEM (more popular)
            1. Clean system
2. BEM - Block Element Modifier

```
.block {}
.block__element {}
.block__element--modifier {}
```

    1. Low specificity BEM selectors
        1. We always use classe, and they are not nested - always low specificity
            1. Hence easy to maintain and re-usable code
    2. Block: Standalone component that is meaningful on its own
        1. Components are the blocks (standalone components that we can use anywhere in the project)
            1. Example:
                1. `recipe`
                2. `btn` (inside `recipe` block)
    3. Element: Part of a block that has no standalone meaning (if we take them out, they will not have any meaning)
        1. `__info`
        2. `__stats-box` (`recipe__stats--box`)
    4. Modifier: A different version of a block or an element
        1. To make it different from regular blocks or elements
            1. Example: To make a button round

```
btn--round
```

                1. Modifier is used to make a different button
                2. `btn` applies to all buttons
3. Example:

```
<figure class="recipe">
    <div class="recipe__hero">
        <img class="recipe__img" src="" />
    </div>
    <div class="recipe__info">
        <div class="recipe__category">
            Veggie
        </div>
        <figoption class="recipe__details">
            <h2 class="recipe__title">Pizza Vegetole</h2>
            <p class="recipe__description">
                Yummy veggie pizza with tasty olives
            </p>
        </figoption>
        <div class="recipe__stats-box">
            <div class="recipe__stat">
                <span class="recipe__stat-value">4.9</span>
                <span class="recipe__stat-name recipe__state-name--1">Stats</span>
```

```
            </div>
            <div class="recipe__stat">
                <span class="recipe__stat-value">45</span>
                <span class="recipe__stat-name recipe__state-name--1">Minutes</span>
            </div>
            <div class="recipe__stat">
                <span class="recipe__stat-value">2</span>
                <span class="recipe__stat-name recipe__state-name--1">Persons</span>
            </div>
        </div>
    </div>
    <a class="recipe__btn btn btn--round" href="#">Try</a>
</figure>
```

1. BEM might look ugly and hard to write (opinion) - more code
    1. It is worth it

Architecting with Files and Folders

1. Strategies
    1. ITCSS
    2. SMAXS
    3. 7-1 pattern
2. 7-1 pattern
    1. 7 different folders for partial Sass files (or any other CSS preprocessor), and
        1. `base/`
            1. Basic project definitions
        2. `components/`
            1. One file for each component
        3. `layout/`
            1. Definition of the overall layout of the project
        4. `pages/`
            1. Styles for specific pages of the project
        5. `themes/`
            1. To implement different visual themes
        6. `abstracts/`
            1. Code that does not output any CSS
                1. Variables
                2. Mixins
                3. ...
        7. `vendors/`
            1. All third party CSS goes
    2. 1 main Sass file to import all other files into a compiled CSS stylesheet
3. We don't have to use all the folders
    1. Depends on size and scope of project

Implementing BEM in the Natours Project

1. What you will learn in this lecture
    1. How to use the BEM method in practice
2. Code: style.css

```
/*
COLORS:

Light green: #7ed56f
Medium green: #55c57a
Dark green: #28b485

*/

html {
  /* font-size: 10px; */ /* 1rem = 10px */
  font-size: 62.5%;
}

* *::after *::before {
  /* after and before don't get the style by default */
  margin: 0;
  padding: 0;
  /* box-sizing: border-box; */
  box-sizing: inherit; /* Inherits whatever we put inside `body` which is `box-sizing: border-box` */
}

body {
  font-family: "Lato", sans-serif;
  font-weight: 400;
  /* font-size: 16px; */
  line-height: 1.7;
  color: #777;
  /* padding: 30px; /* Not inherited */
  padding: 4rem;

  box-sizing: border-box; /* makes it easier to change in plugins. Better practicing */
}

.header {
  height: 95vh; /* 95% of viewport height */
  background-image: linear-gradient(
      to right bottom,
      rgba(126, 213, 111, 0.8),
      rgba(40, 180, 133, 0.8)
    ),
    url("../img/hero.jpg");
```

```
  background-size: cover; /* tries to fit image inside the box */
  background-position: top; /* top of the image stays on top */
  position: relative;

  clip-path: polygon(0 0, 100% 0, 100% 75vh, 0 100%);
  /* clip-path: polygon(50% 0, 100% 100%, 0 100%); */
}

.header__logo-box {
  position: absolute;
  top: 4rem;
  left: 4rem;
}

.header__logo {
  height: 3.5rem;
}

.header__text-box {
  position: absolute;
  top: 50%; /* In relation to the parent element */
  left: 50%;
  transform: translate(
    -50%,
    -40%
  ); /* Relative to the element and not it's parent */
  text-align: center;
}

.heading-primary {
  color: #fff;
  text-transform: uppercase;

  backface-visibility: hidden; /* back part of the element will not be visible. Fixes shaking problem. Hack */

  margin-bottom: 6rem;
}

.heading-primary--main {
  display: block;
  font-size: 6rem;
  font-weight: 400;
  letter-spacing: 3.5rem;

  animation-name: moveInLeft;
  animation-duration: 1s;
  animation-timing-function: ease-out;

  /* animation-iteration-count: 3; */ /* Happens 3 times */
  /* animation-delay: 3s; */
}

.heading-primary--sub {
  display: block;
  font-size: 2rem;
  font-weight: 700;
  letter-spacing: 1.75rem;

  animation: moveInRight 1s ease-out;
}

@keyframes moveInLeft {
  0% {
    opacity: 0;
    transform: translateX(-10rem);
  }

  80% {
    transform: translateX(1rem);
  }

  100% {
    opacity: 1;
    transform: translateX(0);
  }
}

@keyframes moveInRight {
  0% {
    opacity: 0;
    transform: translateX(10rem);
  }

  80% {
    transform: translateX(-1rem);
  }

  100% {
    opacity: 1;
    transform: translateX(0);
  }
}

@keyframes moveInBotton {
  0% {
    opacity: 0;
    transform: translateY(3rem);
  }

  100% {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
.btn:link,
.btn:visited {
  text-transform: uppercase;
  text-decoration: none;
  padding: 1.5rem 4rem;
  display: inline-block;
  border-radius: 10rem;
  transition: all 0.2s;
  position: relative;
  font-size: 1.6rem;
}

.btn:hover {
  transform: translateY(-0.3rem);
  box-shadow: 0 1rem 2rem rgba(0, 0, 0, 0.2); /* 0px - x axis, 10px down - y axis, 20px - blur  */
}

.btn:active {
  transform: translateY(-0.1rem); /* in relation to original state */
  box-shadow: 0 5px 10px rgba(0, 0, 0, 0.2);
}

.btn--white {
  background-color: #fff;
  color: #777;
}

/* Acts like a child of `btn` */
.btn::after {
  content: "";
  display: inline-block;
  height: 100%; /* same height as parent: `btn` */
  width: 100%;
  border-radius: 10rem;
  position: absolute;
  top: 0;
  left: 0;
  z-index: -1; /* Behind the button */
  transition: all 0.4s;
}

.btn--white::after {
  background-color: #fff;
}

/* On hover of button apply style to pseudo element ::after */
.btn:hover::after {
  transform: scaleX(1.5) scaleY(1.6);
  opacity: 0;
}

.btn--animated {
  animation: moveInBotton 0.5s ease-out 0.75s;
  animation-fill-mode: backwards; /* It automatically applies the style of 0% before the animation starts */
}
```

Section 4: Introduction to Sass and NPM

Section Intro

1. Sass
    1. Basics
    2. Not hard to learn
        1. Extension of CSS
    3. Sass compiler
        1. NPM ecosystem
            1. How to compile using NPM packages & NPM scripts

What is Sass?

What is Sass and How Does it Work?

1. Sass is a CSS preprocessor, an extension of CSS that adds power and elegance to the basic language
    1. SASS Source Code -Sass compiler-> Compiled CSS Code
2. Sass is used to fix problems with CSS
    1. CSS can get very messy very quickly
    2. CSS has no re-usable pieces
    3. CSS has no logic
3. Sass has handy features
4. How does it work?
    1. We write Sass code in Sass files
    2. Sass compiler converts Sass code to CSS code
5. Website doesn't know Sass
    1. It only sees CSS

Main Sass Features

1. **Variables:** for reusable values such as colors, font-size, spacing, etc.
2. **Nesting:** to nest selectors inside of one another, allowing us to write less code
3. **Operators:** for mathematical operations right inside of CSS
4. **Partials and imports:** to write CSS in different files and importing them all into one single file
5. **Mixins:** to write reusable pieces of CSS code
6. **Functions:** similar to mixins, with the difference that they produce a value that can then be used

7. **Extends:** to make different selectors inherit declarations that are common to all of them
8. **Control directives:** to write complex code using conditionals and loops (not covered in this course)

Sass and SCSS: Clearing up the Confusion

1. Sass
    1. SASS syntax
        1. Indentation sensitive and doesn't use curly braces or semi-colons
            1. Confusing
            2. Difficult to learn
    2. SCSS syntax (Sassy CSS)
        1. Preserves the way original CSS looks like
2. Code: index.html

```
<nav>
  <ul class="navigation">
    <li><a href="#">About Us</a></li>
    <li><a href="#">Pricing</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
  <div class="buttons">
    <a class="btn-main" href="#">Sign Up</a>
    <a class="btn-hot" href="#">Get a Quote</a>
  </div>
</nav>
```

3. Code: style.css

```
* {
  margin: 0;
  padding: 0;
}

$color-primary: #f9ed69; // yellow
$color-secondary: #f08a5d; // orange
$color-tertiary: #b83b5e; // pink
$color-text-dark: #333;
$color-text-light: #eee;

$width-button: 150px;

nav { // collapses - height = 0
  margin: 30px;
  background-color: $color-primary;

  &::after {
    content: "";
    clear: both;
    display: table;
  }
}

.navigation {
  list-style: none;
  float: left;

  li {
    display: inline-block;
    margin-left: 30px;

    &:first-child { // .navigation li:first-child
      margin: 0;
    }

    a:link {
      text-decoration: none;
      text-transform: uppercase;
      color: $color-text-dark;
    }
  }
}

.buttons {
  float: right;
}

.btn-main:link,
.btn-hot:link {
  padding: 10px;
  display: inline-block;
  text-align: center;
  border-radius: 100px;
  text-decoration: none;
  text-transform: uppercase;
  width: $width-button;
  color: $color-text-light;
}

.btn-main {
  &:link {
    background-color: $color-secondary;
  }

  &:hover {
    background-color: darken($color-secondary, 15%);
  }
}

.btn-hot {
```

```
  &:link {
    background-color: $color-tertiary;
  }

  &:hover {
    background-color: lighten($color-secondary, 10%);
  }
}
```

First Steps with Sass: Variables and Nesting

1. Mixin:
    1. A re-usable piece of code we write in a mixin
        1. The code in mixin is used in the place where we called the mixin
            1. Like a variable with multiple lines of code

First Steps with Sass: Mixins, Extends, and Functions

1. Code: style.css

```
* {
  margin: 0;
  padding: 0;
}

$color-primary: #f9ed69; // yellow
$color-secondary: #f08a5d; // orange
$color-tertiary: #b83b5e; // pink
$color-text-dark: #333;
$color-text-light: #eee;

$width-button: 150px;

@mixin clearfix {
  &::after {
    content: "";
    clear: both;
    display: table;
  }
}

@mixin style-link-text($color) {
  text-decoration: none;
  text-transform: uppercase;
  color: $color;
}

@function divide($a, $b) {
  @return $a / $b;
}

nav { // collapses - height = 0
  margin: divide(60, 2) * 1px; // 30px
  background-color: $color-primary;

  @include clearfix;
}

.navigation {
  list-style: none;
  float: left;

  li {
    display: inline-block;
    margin-left: 30px;

    &:first-child { // .navigation li:first-child
      margin: 0;
    }

    a:link {
      @include style-link-text($color-text-dark);
    }
  }
}

.buttons {
  float: right;
}

%btn-placeholder { // A selector is copied to the placeholder with @extend
  padding: 10px;
  display: inline-block;
  text-align: center;
  border-radius: 100px;
  width: $width-button;

  @include style-link-text($color-text-light);
}

.btn-main {
  &:link {
    @extend %btn-placeholder;
    background-color: $color-secondary;
  }

  &:hover {
    background-color: darken($color-secondary, 15%);
  }
}
```

```
.btn-hot {
  &:link {
    @extend %btn-placeholder;
    background-color: $color-tertiary;
  }

  &:hover {
    background-color: lighten($color-secondary, 10%);
  }
}
```

A Brief Introduction to the Command Line

1. Install and compiling Sass
2. Windows:
    1. Command Prompt
        1. `dir`
        2. `cd <folder>`
        3. `cd..`
        4. `cls`
        5. `mkdir`
        6. `move`
        7. `rm/del`
            1. `rm` deletes permanently
            2. `rm -r` - recursive

NPM Packages: Let's Install Sass Locally

1. Installing Sass on local computer

A Brief Introduction to NPM and the Node Ecysystem

1. Node.js and NPM package ecosystem
    1. Tools
    2. Libraries
    3. Frameworks
2. Node.js
    1. Allows developers to write and run JS applications on the server. Developers started using node.js to also write tools to help them with **local web development**
3. NPM
    1. NPM is a simple command line interface that allows developers to **install and manage packages** on their local computers. There are all kinds of open-source **tools, libraries, and frameworks** needed for modern development. Modern web development could simply not exist without a package manager.
        1. It is also used to automate boring tasks
4. Install node.js
5. Initialize npm

    ```
    npm init
    ```

6. Install Sass

    ```
    npm init node-sass --save-dev # **(M)**
    ```

7. Re-install dependencies

    ```
    npm install
    ```

8. Uninstall a package

    ```
    npm uninstall jquery --save
    ```

NPM Scripts: Let's Write and Compile Sass Locally

1. package.json

    ```
    "scripts": {
      "compile:sass": "node-sass sass/main.scss css/style.css"
    },
    ```

    1. Other packages
        1. Package to prefix code to support all browsers
        2. Package to compress code
        3. ...
    2. Watch mode:

        ```
        "scripts": {
          "compile:sass": "node-sass sass/main.scss css/style.css -w"
        },
        ```

The Easiest Way of Automatically Reloading a Page on File Changes

1. `npm i live-server -g` - for all future projects and anywhere in the computer
2. Running live server

1. `cd starter`
2. `live-server`

Section 5: Natours Project - Using Advanced CSS and Sass (Part 2)

Section Intro

1. Continue working on the Natours project

Converting Our CSS Code to Sass: Variables and Nesting

1. Converting CSS code into Sass Code
2. All selectors have same specificity and it is low
    1. It is hard to nest because most elements have classes
3. Code: main.scss

```
$color-primary: #55c57a;
$color-primary-light: #7ed56f;
$color-primary-dark: #28b485;

$color-gray-dark: #777;
$color-white: #fff;
$color-black: #000;

html {
  /* font-size: 10px; */ /* 1rem = 10px */
  font-size: 62.5%;
}

* *::after *::before {
  /* after and before don't get the style by default */
  margin: 0;
  padding: 0;
  /* box-sizing: border-box; */
  box-sizing: inherit; /* Inherits whatever we put inside `body` which is `box-sizing: border-box` */
}

body {
  font-family: "Lato", sans-serif;
  font-weight: 400;
  /* font-size: 16px; */
  line-height: 1.7;
  color: $color-gray-dark;
  /* padding: 30px; /* Not inherited */
  padding: 4rem;

  box-sizing: border-box; /* makes it easier to change in plugins. Better practicing */
}

.header {
  height: 95vh; /* 95% of viewport height */
  background-image: linear-gradient(
      to right bottom,
      rgba($color-primary-light, 0.8),
      rgba($color-primary-dark, 0.8)
    ),
    url("../img/hero.jpg");
  background-size: cover; /* tries to fit image inside the box */
  background-position: top; /* top of the image stays on top */
  position: relative;

  clip-path: polygon(0 0, 100% 0, 100% 75vh, 0 100%);
  /* clip-path: polygon(50% 0, 100% 100%, 0 100%); */

  &__logo-box {
    position: absolute;
    top: 4rem;
    left: 4rem;
  }

  &__logo {
    height: 3.5rem;
  }

  &__text-box {
    position: absolute;
    top: 50%; /* In relation to the parent element */
    left: 50%;
    transform: translate(
      -50%,
      -40%
    ); /* Relative to the element and not it's parent */
    text-align: center;
  }
}

.heading-primary {
  color: $color-white;
  text-transform: uppercase;

  backface-visibility: hidden; /* back part of the element will not be visible. Fixes shaking problem. Hack */

  margin-bottom: 6rem;

  &--main {
    display: block;
```

```
      font-size: 6rem;
      font-weight: 400;
      letter-spacing: 3.5rem;

      animation-name: moveInLeft;
      animation-duration: 1s;
      animation-timing-function: ease-out;

      /* animation-iteration-count: 3; */ /* Happens 3 times */
      /* animation-delay: 3s; */
    }

    &--sub {
      display: block;
      font-size: 2rem;
      font-weight: 700;
      letter-spacing: 1.75rem;

      animation: moveInRight 1s ease-out;
    }
  }

  @keyframes moveInLeft {
    0% {
      opacity: 0;
      transform: translateX(-10rem);
    }

    80% {
      transform: translateX(1rem);
    }

    100% {
      opacity: 1;
      transform: translateX(0);
    }
  }

  @keyframes moveInRight {
    0% {
      opacity: 0;
      transform: translateX(10rem);
    }

    80% {
      transform: translateX(-1rem);
    }

    100% {
      opacity: 1;
      transform: translateX(0);
    }
  }

  @keyframes moveInBotton {
    0% {
      opacity: 0;
      transform: translateY(3rem);
    }

    100% {
      opacity: 1;
      transform: translateY(0);
    }
  }

  .btn {
    &:link,
    &:visited {
      text-transform: uppercase;
      text-decoration: none;
      padding: 1.5rem 4rem;
      display: inline-block;
      border-radius: 10rem;
      transition: all 0.2s;
      position: relative;
      font-size: 1.6rem;
    }

    &:hover {
      transform: translateY(-0.3rem);
      box-shadow: 0 1rem 2rem rgba($color-black, 0.2); /* 0px - x axis, 10px down - y axis, 20px - blur  */ /* Works only in Sass */

      /* On hover of button apply style to pseudo element ::after */
      &::after {
        transform: scaleX(1.5) scaleY(1.6);
        opacity: 0;
      }
    }

    &:active {
      transform: translateY(-0.1rem); /* in relation to original state */
      box-shadow: 0 5px 10px rgba($color-black, 0.2);
    }

    &--white {
      background-color: $color-white;
      color: $color-gray-dark;

      &::after {
        background-color: $color-white;
      }
    }
```

```
        /* Acts like a child of `btn` */
        &::after {
          content: "";
          display: inline-block;
          height: 100%; /* same height as parent: `btn` */
          width: 100%;
          border-radius: 10rem;
          position: absolute;
          top: 0;
          left: 0;
          z-index: -1; /* Behind the button */
          transition: all 0.4s;
        }

        &--animated {
          animation: moveInBotton 0.5s ease-out 0.75s;
          animation-fill-mode: backwards; /* It automatically applies the style of 0% before the animation starts */
        }
      }
```

Implementing the 7-1 CSS Architecture with Sass

1. The architecture is required for big projects with multiple pages
    1. themes
        1. Web app with different themes
    2. vendors
        1. Third party CSS
            1. CSS file of Bootstrap
            2. Icon system
            3. Animation framework

Review: Responsive Design Principles and Layout Types

Review: Basic Responsive Design Principles

1. Technique to adjust the layout to any possible screen size
    1. screen size: Just the window/viewport size
    2. Makes website usable on all possible devices
2. Four Web design principles (big important ones)
    1. Fluid Layouts (this needs to be implemented from the beginning)
        1. To allow webpage to adapt to the current viewport width (or even height)
        2. Use % (or vh/vw) unit instead of px for elements that should adapt to viewport (usually layout)
        3. Use `max-width` instead of `width`
    2. Responsive Units
        1. Use `rem` unit instead of `px` for most lengths (in CSS)
        2. To make it easy to scale the entire layout down (or up) automatically
    3. Flexible Images
        1. By default, images don't scale automatically as we change the viewport, so we need to fix that
        2. Always use % for image dimensions, together with the `max-width` property
    4. Media Queries (important only if the above 3 are done)
        1. To change CSS styles on certain viewport widths (called breakpoints)
            1. Different versions of the website for different devices

Layout Types

1. Types
    1. Float Layouts
        1. The **old way of building layouts** of all sizes, using the float CSS property. Still used, but getting outdated.
            1. Placing boxes side by side using `float` property
    2. Flexbox
        1. Modern way of laying out elements in a **1-dimensional row** wihout using floats. Perfect for **component layouts**
    3. CSS Grid
        1. For laying out element in a fully-fledged **2-dimensional grid**. Perfect for **page layouts and complex components**

Why Use a Float Layout in This Project?

1. The focus of this project is on using modern CSS properties and techniques: clips, transforms, animations, background video, etc.
    1. We will build a float-based custom grid, and use it throughout the project
2. Even though flexbox and CSS grid are more modern, **every web developer should still know how float layouts work**
    1. Reasons:
        1. As a professional developer, **your job will include working with older CSS codebases**, which

Building a Custom Grid with Floats

What You Will Learn in this Lecture

1. How to architect and build a simple grid system.
2. How the attribute selector works
3. How the `:not` pseudo-class works
4. How `calc()` works, and what's the difference between `calc()` and simple Sass operations

What is a Grid

1. Design system to build consistent interfaces

    1. Equal parts
    2. 1/3rd and 2/3rds

3. etc.
4. Consistent spacing
    1. Gutters

2. Configurations

    1. Two columns
    2. Three columns
    3. Four columns
        1. 1 + 1 + 1 + 1
        2. 1 + 1 + 2
        3. 1 + 3

3. We need rows as well

4. Code: _grid.scss

```
.row {
  max-width: $grid-width; // 100% width if vw is < 114rem else 114rem
  background-color: #eee;
  margin: 0 auto; // Browser will figure out the left and right margins

  &:not(:last-child) {
    margin-bottom: $gutter-vertical;
  }

  @include clearfix;

  [class^="col-"] {
    background-color: orangered;
    float: left;

    &:not(:last-child) {
      margin-right: $gutter-horizontal;
    }
  }

  .col-1-of-2 {
    width: calc(
      (100% - #{$gutter-horizontal}) / 2
    ); // We can mix units. Not possible in CSS. Calculation depends on layout
  }

  .col-1-of-3 {
    width: calc((100% - 2 * #{$gutter-horizontal}) / 3);
  }

  .col-1-of-4 {
    width: calc((100% - 3 * #{$gutter-horizontal}) / 4);
  }

  .col-2-of-3 {
    width: calc(
      2 * ((100% - 2 * #{$gutter-horizontal}) / 3) + #{$gutter-horizontal}
    );
  }

  .col-2-of-4 {
    width: calc(
      2 * ((100% - 3 * #{$gutter-horizontal}) / 4) + #{$gutter-horizontal}
    );
  }

  .col-3-of-4 {
    width: calc(
      3 * ((100% - 3 * #{$gutter-horizontal}) / 4) + 2 * #{$gutter-horizontal}
    );
  }
}
```

5. Code: _variables.scss

```
// GRID
$grid-width: 114rem;
$gutter-vertical: 8rem;
$gutter-horizontal: 6rem;
```

6. Code: index.html

```
<section class="grid-test">
    <div class="row">
        <div class="col-1-of-2">
            Col 1 of 2
        </div>
        <div class="col-1-of-2">
            Col 1 of 2
        </div>
    </div>
    <div class="row">
        <div class="col-1-of-3">
            Col 1 of 3
        </div>
        <div class="col-1-of-3">
            Col 1 of 3
        </div>
        <div class="col-1-of-3">
            Col 1 of 3
        </div>
    </div>
    <div class="row">
        <div class="col-1-of-3">
```

```
                Col 1 of 3
            </div>
            <div class="col-2-of-3">
                Col 2 of 3
            </div>
        </div>
        <div class="row">
            <div class="col-1-of-4">
                Col 1 of 4
            </div>
            <div class="col-1-of-4">
                Col 2 of 4
            </div>
            <div class="col-1-of-4">
                Col 3 of 4
            </div>
            <div class="col-1-of-4">
                Col 4 of 4
            </div>
        </div>
        <div class="row">
            <div class="col-1-of-4">
                Col 1 of 4
            </div>
            <div class="col-1-of-4">
                Col 2 of 4
            </div>
            <div class="col-2-of-4">
                Col 2 of 4
            </div>
        </div>
        <div class="row">
            <div class="col-1-of-4">
                Col 1 of 4
            </div>
            <div class="col-3-of-4">
                Col 3 of 4
            </div>
        </div>
    </section>
```

Building the About Section - Part 1

What you will Learn in this Lecture

1. Think about components
    1. Where to place the code in the overall CSS architecture
2. How and why to use utility classes
3. How to use the `background-clip` property
4. How to `transform` multiple properties simultaneously
5. How to use the `outline-offset` property together with `outline`
6. How to style elements that are NOT hovered while others are

Building the About Section - Part 2

1. We can see computed values in Dev Tools
    1. Computed
        1. Shows all the properties with values

Building the About Section - Part 3

Building the Features Section

What You Will Learn in this Lecture

1. How to include and use an icon font
2. Another way of constructing the "skewed section" design
3. How and when to use the direct child selector
4. linea.io
    1. Download basic icons
        1. Copy and paste from `C:\Users\suppo\Downloads\linea_basic_1.0\_ICONFONT`

Building the Tours Section - Part 1

What you will Learn in this Lecture

1. How to build an amazing, rotating card
2. How to use `perspective` in CSS
3. How to use the `backface-visibility` property
4. Using background blend modes
5. How and when to use `box-decoration-break`

Building the Tours Section - Part 2

1. Images:
    1. unsplash.com - free images
2. `background-blend-mode` - how background images should blend

Building the Tours Section - Part 3

1. `clip-path` breaks the `overflow` property

Building the Stories Section - Part 1

What you will Learn in this Lecture

1. How to make text flow around shapes with `shape-outside` and `float`
2. How to apply a `filter` to images
3. How to construct a background video covering an entire section
4. How to use the `<video` HTML element
5. How and when to use the `object-fit` property

Building the Stories Section - Part 2

Building the Stories Section - Part 3

1. Free videos: https://coverr.co

Building the Booking Section - Part 1

What you will learn in this lecture

1. How to implement "solid-color gradients"
2. How the general and adjacent sibling selectors work and why we need them
3. How to use the `::input-placeholder` pseudo-element
4. How and when to use the `:focus`, `:invalid`, `placeholder-shown`, and `:checked` pseudo-classes
5. Techniques to build custom radio buttons

Building the Booking Section - Part 2

1. Browsers do not give font family of the page to the form elements by default.
    1. We have to explicitly inherit the font family.

        ```
        font-family: inherit
        ```

2. Google chrome validation
    1. Tests if we are writing valid inputs in form (email, etc...)
        1. CSS has `invalid` or `valid` pseudo-classes
3. To make placeholder visible:

```
&__input:placeholder-shown + &__label {
  // + : adjacent sibling (after), ~ : general sibling
  opacity: 0; // element will still be on the page
  visibility: hidden; // removed from the page. We cannot animate it though
  transform: translateY(-4rem);
}
```

Building the Booking Section - Part 3

1. There is no way to style radio buttons in CSS
    1. Trick:
        1. Add another button in `label`
        2. Hide the `input`
            1. If the label is clicked, the `input` gets selected because they are associated
2. Similar for checkboxes
3. Utility class can have `!important` to override other styles

Building the Footer

What you will Learn in the Lecture

1. How to design a simple website footer

Building the Navigation - Part 1

What You will Learn in This Lecture

1. What the "checkbox hack" is and how it works
2. How to construct custom animation timing functions using cubic bezier curves
3. How to animate "solid-color gradients"
4. How and why to use `transform-origin`
5. In general: Construct an amazingly creative effect

Building the Navigation - Part 2

1. Cubic Bezier functions
    1. https://easigs.net
    2. https://cubic-bezier.com

Building the Navigation - Part 3

Building a Pure CSS Popup - Part 1

What you will Learn in This Lecture

1. How to build a nice popup with only CSS
2. How to use the `:target` pseudo-class
3. How to construct boxes with equal height using `display: table-cell`
4. How to construct CSS text columns
5. How to automatically hyphenate words using hyphens

Building a Pure CSS Popup - Part 2

Section 6: Natours Project - Advanced Responsive Design (Part 3)

Section Intro

1. Advanced responsive concepts
    1. Media queries
    2. Screen widths
    3. Resolutions
    4. Different touch capabilities
    5. Responsive images
    6. Feature queries
        1. For browsers to with different capabilities

Mobile-First vs Desktop-First and Breakpoints

Responsive Design Strategies

1. Desktop-first
    1. Steps:
        1. Start writing CSS for the desktop: large screen
        2. Then, media queries shrink design to smaller screens
    2. Traditional way
        1. The easier way to learn
            1. Writing media queries to test for `max-width`

```
html { font-size: 20px; }
@media (max-width: 600px) {
    html { font-size: 16px; }
}
```

2. Mobile-first
    1. Steps:
        1. Start writing CSS for mobile devices: small screen
        2. Then, media queries expand design to a large desktop screen
    2. Advantages:
        1. Forces us to reduce websites and apps to the absolute essentials (for smaller and faster final product)

```
html { font-size: 16px; }
@media (min-width: 600px) {
    html { font-size: 20px; }
}
```

Responsive Design Strategies: Max-Width and Min-Width

1. Pixel spectrum:

```
0px --- 600px --- 900px --- 1200px --->
```

    1. Dektop-first
        1. Our initial **desktop-first** code goes on the right side
        2. `max-width` - checks if the current viewport width (vw) is <= 600px
            1. If yes,
                1. All the code in the media query will apply
            2. If no,
                1. All the code in the media query will not apply
        3. Media queries
            1. Used to override specific parts that we want to change
        4. If a phone size is such that multiple media queries match, then which one applies?
            1. Both apply
                1. Media queries don't add any importance or specificity to selectors, so code order matters - **media queries at the end**
                    1. **The one that appears last will take presedence** (order of the code matters)
                        1. Hence media queries are kept at the end
    2. Mobile-first
        1. `min-width`
            1. Is the current viewport width >= 600px?
                1. If yes, the code applies
                2. If no, the code doesn't apply
        2. Reason for this approach?
            1. We want media queries to stay away from smaller screen styles

Is Mobile-First Right for You?

1. Pros
    1. 100% optimised for the mobile experience
    2. Reduces websites and apps to the absolute essentials
    3. Results in smaller, faster and more efficient products
        1. Optimized for mobile users
    4. Prioritizes content over aesthetic design, which may be desirable
        1. Content is king
2. Cons
    1. The desktop version might feel overly empty and simplistic
        1. Why have a website for desktop if it doesn't use the features and capabilities of the desktop computer?
    2. More difficult and counterintuitive to desktop
        1. Larger canvas has less restriction
            1. Gives a lot of freedom for design
    3. Less creative freedom, making it more difficult to construct distinctive products
    4. Clients are used to see a desktop version of the site as a prototype (not just mobile version)
    5. Do your users even use the mobile internet? What's the purpose of your website?
        1. They might be using desktop or laptop
3. Advice:
    1. No matter what we do, always keep both desktop and mobile in mind (both are important)
        1. Don't leave the other as afterthought

Selecting Our Breakpoints: The Options

1. Bad
    1. The most used way
        1. Using widths of popular devices (iphones, ipads, ...)
            1. Problems
                1. Optimizing for specific devices ignoring the users of other devices (maybe the others are more used)
                2. Not future-proof
                    1. If Apple decides to change their resolutions
                        1. We will end up changing media queries
2. Good
    1. We use all the most used device widths on the internet & try to group them together
    2. Pick breakpoints from that
        1. In between the groups
    3. Better
        1. A lot of devices are used
        2. Not setting breakpoints at specific widths but between similar devices
3. Perfect
    1. Ignore all the devices & only look at the content of the design
        1. Begin at one size
        2. Start increasing / decreasing screen widths
        3. As soon as design breaks (it doesn't look okay) insert breakpoint
    2. Problems
        1. It can be extremely difficult to implement
        2. It is had to find the best breakpoints
        3. We may end up with many breakpoints

Setting Our Breakpoints: A Good Approach

1. Scale: http://gs.statcounter.com/screen-resolution-stats#monthly-201608-201708-bar
    1. 0px
    2. 200px
    3. 400px
    4. 600px
    5. 800px
    6. 1200px
    7. 1400px
    8. 1600px
    9. 1800px
    10. 2000px
2. We need breakpoints for
    1. Phones
    2. Portrait tablets
    3. Landscape tablets
    4. Desktop
3. Breakpoints:
    1. Phone only: <= 600px (iPhone)
    2. Portrait tablet: <= 900px (iPad)
    3. Landscape tablet: <= 1200px (iPad)
    4. Desktop: <= 1800px (Macbook Pro 15")
    5. Big desktop: > 1800px (optional - huge screens)

Let's Use the Power of Sass Mixins to Write Media Queries

What You will Learn in This Lecture

1. How to use a powerful Sass mixin to write all our media queries
2. How to use the `@content` and `@if` Sass directives
3. Taking advantage of Chrome DevTools for responsive design

Better way to Write Media Queries - Mixin

1. `rems` and `ems` are not affected by root font size
    1. 1 em = 1 rem = font-size coming from browser (16px)

1. `rems` fail to work in some of the browsers
    1. `ems` are he best option for media queries

Writing Media Queries - Base, Typography and Layout

Writing Media Queries - Layout, About and Features Sections

Writing Media Queries - Tours, Stories and Booking Sections

1. App: https://sizzy.co
    1. If we enter URL, it will show pages on a lot of devices

An Overview of Responsive Images

What are Responsive Images Anyway?

1. They are important for both responsive design and web-performance
2. Responsive Images
    1. The goal of responsive images is to serve the **right image** to the **right screen size** and device, in order to avoid downloading unnecessary large images on smaller screens
        1. 1MB - Desktop
        2. 200kb - kb (slow/expensive data plan anywhere in the world)

When to Use Responsive Images: The 3 Use Cases

1. Resolution switching
    1. Large screen -> Small screen
        1. Decrease image resolution on smaller screen (same image with smaller resolution)
2. Density switching (screen size doesn't matter but screen's pixel density matters)
    1. @2x screen (high-res) -> @1x screen (low-res)
        1. Half the image resolution on @1x screen
        2. Pixel density: Number of pixels found in an inch or cm
            1. Low resolution screens: PC screens
                1. 1x screen - They use 1px to display 1px of the design
                    1. If we specify 100px as height, they use 100 physical pixels to display the 100px we specified
            2. High resolution screens: Smart phones, some computers (Mac Books with retina display)
                1. 2x screens - They use two physical pixels to display 1px of our design
                    1. If we specify 100px as height, they use 200 physical pixels to display the 100px we specified
                        1. => **We need to serve an image with double the resolution of the original image**
3. Art direction
    1. Large screen -> small screen
        1. Different image on smaller screen
            1. Image details were preserved, but the image is different (clipped)
                1. We might load a different image (clipped and smaller)
4. Why discuss?
    1. We need to choose different solutions to different use-cases

Responsive Images in HTML - Art Direction and Density Switching

What You will Learn in this Lecture

1. How to use the `srcset` attribute on the `<img>` and `<source>` elements, together with density descriptors
2. How and why to use the `<picture>` element for art direction (with media queries in HTML)
3. How to write media queries in HTML

Implementation

1. There are more technologies to do responsive images in HTML

2. Density descriptors

```
<img srcset="img/logo-green-1x.png 1x, img/logo-green-2x.png 2x" alt="Full logo" class="footer__logo">
```

3. Making images depend on screen width - art direction

```
<picture class="footer__logo"> <!--Art direction -->
    <source srcset="img/logo-green-small-1x.png 1x, img/logo-green-small-2x.png 2x"
        media="(max-width: 37.5em)"> <!--Density switching -->
    <!-- 0 or more sources, media query  -->
    <img srcset="img/logo-green-1x.png 1x, img/logo-green-2x.png 2x" alt="Full logo" class="footer__logo">
    <!-- Fallback, default -->
</picture>
```

Responsive Images in HTML - Density and Resolution Switching

What You will Learn In This Lecture

1. Density and Resolution Switching: How to allow the browser to decide the best image to download, using the `srcset` attribute, width descriptors, and the `sizes` attribute of the `<img>` element
2. We can set device pixel ratio (DPR - 1.0 or 2.0)

Responsive Images in CSS

What You will Learn in this Lecture

1. How to implement responsive images in CSS
2. How to use resolution media queries to target high-resolution screens with `2x`
3. How to combine multiple conditions in media queries

Implementation

1. We just have to write media queries to load different images in different situations
    1. Based on
        1. Viewport width
        2. Screen resolution
2. Code: _header.scss

```
@media (min-resolution: 192dpi) and (min-width: 37.5em), (min-width: 125em) {
  // **(M)** resolution of Apple retina screen (reference)
  background-image: linear-gradient(
      to right bottom,
      rgba($color-primary-light, 0.8),
      rgba($color-primary-dark, 0.8)
    ),
    url("../img/hero.jpg");
}
```

Testing for Browser Support with @supports

Warning

1. Many of the new CSS features shown are highly experimental and only work in top modern browsers
    1. Always check caniuse.com before using a modern CSS poperty in production
    2. Use graceful degradation with `@supports`
        1. Top notch experience for modern browsers, and reduced experience for other browsers

What You will Learn in This Lecture

1. How to use `@supports` feature queries
2. Implement graceful degradation on selected properties
3. How to use `backgrop-filter`

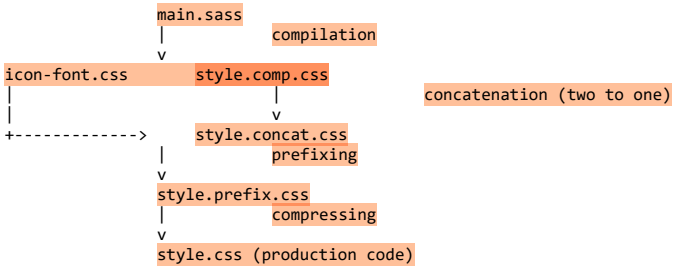
backdrop-filter

1. It applies a filter to what's behind the selected element
    1. Example: Blurring instead of making it dark

```
@supports (-webkit-backdrop-filter: blur(10px)) or
  (backdrop-filter: blur(10px)) {
  -webkit-backdrop-filter: blur(10px);
  backdrop-filter: blur(
    10px
  ); // **(M)**. Works only in Safari. It works in Chrome now.
}
```

Setting up a Simple Build Process with NPM Scripts

A Simple Build Process



1. Automatically adding prefixes
2. Compressing code
3. Build process
    1. Sequence of tasks done automatically after we are done developing the product/feature
        1. The final output is ready to be deployed to production

Wrapping up the Natours Project: Final Considerations

1. Effect that happens when we select text
2. Media query:

```
@media only screen and (max-width: 37.5em) {
    // ...
}
```

3. In CSS we can identify touch devices using media query

```
// @include respond(tab-port) {
@media only screen and (max-width: 56.25em), only screen and (hover: none) {
```

```
    // ...
}
```

4. Commenting code
    1. A lot of comments are required for real-world
5. Variables
    1. All numbers could be stored in variables

Section 7: Trillo Project - Master Flexbox!

Section Intro

1. Flexbox
    1. A revolution in CSS
2. Topics
    1. Fundamental concepts and properties of flexbox
    2. Demo
    3. Trillo - project
    4. Custom CSS properties
    5. Masks
    6. SVGs

Why Flexbox: An Overview of the Philosophy Behind Flexbox

A True Revolution

1. Flexbox
    1. Flexbox is a new module in CSS3 that makes it easy to align elements to one another, in different directions and orders
        1. Change for responsive design
    2. The main idea behind flexbox is to give the container the ability to expand and to shrink elements to best use all the available space
    3. Flexbox replaces float layouts, using less, and more readable and logical code
    4. Flexbox completely changes the way that we build one-dimensional layouts
        1. There are better ways of building two-demensional layouts
            1. CSS Grid layouts
    5. A true revolution in CSS
        1. Makes it easier for CSS developers

Main Flexbox Concepts

1. `display: flex`
    1. `display: flex-inline`
        1. Flex-container that behaves like an inline element
    2. The element on which we use flex-box is called **flex container**
    3. All the direct children of flex-container are called **flex items**
    4. The direction in which the flex items are layed out is called **main axis**
        1. Direction can be changed
        2. There are different ways to align elements along main axis and cross axis
    5. The perpendicular axis is called **cross axis**

Flexbox Properties Overview

1. Container properties
    1. `flex-direction: row | row-reverse | column | column-reverse`
    2. `flex-wrap: nowrap | wrap | wrap-reverse`
        1. Defines if flex items should wrap into a new line if there is not enough space in a flex container or not
    3. `justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly`
        1. Defines how flex items should be aligned on the main axis
    4. `align-items: stretch | flex-start | flex-end | center | baseline`
        1. Defines how flex items should be aligned along the cross axis
    5. `align-content: stretch | flex-start | flex-end | center | baseline`
        1. Applies only if there is more than one row of flex items
        2. Controls how rows are aligned on the cross axis if there is empty space
2. Item
    1. `align-self: auto | stretch | flex-start | flex-end | center | baseline`
        1. For one individual flex item
    2. `order: 0 | <integer>`
        1. Order in which one specific flex item should appear inside the container
            1. To re-order items - for smaller screens say
    3. `flex-grow: 0 | <integer>`
        1. Decides the width of the flex item
        2. Defines how much the item can grow
    4. `flex-shrink: 1 | <integer>`
        1. Decides the width of the flex item
        2. Defines how much the item can shrink
    5. `flex-basis: auto | <length>`
        1. Decides the width of the flex item
        2. Defines base width
    6. `flex: 0 1 auto | <int> <int> <len>`
        1. Shorthand for the above three properties

A Basic Intro to Flexbox: The Flex Container

1. `stretch` - stretches all items to match the height of the tallest item

2. `baseline` - text are aligned on top of a line
3. `space-between` - 1x, 2x, 2x, 2x, 1x
4. `space-around` - 1x, 1x, 1x, 1x, 1x
5. Code:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

.container {
  background-color: #ccc;
  padding: 10px;
/*   height: 2000px; */

  display: flex;
  flex-direction: row;
  justify-content: space-around;
  align-items: center;
}

.item {
  background-color: #f1425d;
  padding: 40px;
  margin: 30px;
  color: #fff;
  font-size: 40px;
}

.i2 {
  height: 200px;
}

.i4 {
/*   font-size: 70px; */
}


<div class="container">
  <div class="item">1</div>
  <div class="item i2">2</div>
  <div class="item">3</div>
  <div class="item i4">4</div>
  <div class="item">5</div>
</div>
```

A Basic Intro to Flexbox: Flex Items

1. We need to made `order` less than `0` because `0` is the first value

   `order: -1`

   1. Order: `-1 0 0 0 1`
   2. Example:

      ```
      .i2 {
        height: 200px;
        order: 2;
      }

      .i3 {
        order: 1;
      }

      .i4 {
      /*   font-size: 70px; */
        align-self: flex-end;
        order: -1;
      }
      ```

2. `flex-grow` -
   1. `flex-grow: 1` - occupy as much space as they can (it matters in relation to other numbers)
      1. If container has `flex-grow: 1` and and item has `flex-grow: 2`, the item has twice the ability to grow (twice the size of others).
         1. `flex` - shorthand
3. `flex-basis` - instead of `width`
   1. `flex-basis: 20%` - 20% of the container
      1. `auto` - default
         1. Occupies only the space that it needs
      2. if set to `300px` it starts decreasing when there is no more space
         1. Fix: `flex-shrink` - controls how an element will shrink (initial value = 1. Element is allowed to shrink)
            1. `flex-shrink: 0` - no longer allowed to shrink (doesn't change its width)
   2. `flex: 0 1 300px` - `flex-grow flex-shrink flex-basis`

A Basic Intro to Flexbox: Adding More Flex Items

1. Problem: We have many flex items
   1. They will start overflowing (out of the screen)
      1. Solution: `flex-wrap: wrap` (`nowrap` - default)
         1. A new line is created for flex items that no longer fit in the container

2. `align-content: flex-start` - aligns entire rows along cross axis
    1. `align-items: flex-start` - aligns items
    2. `align-content: stretch` - stretches the rows and not items

Project Overview

1. Trillo
    1. Fictional booking app
        1. Hotel
        2. Flight
        3. Car
        4. Tours

Defining Project Settings and Custom Properties

What You Will Learn in This Lecture

1. How and why to use CSS custom properties
2. CSS custom properties (AKA variables)
    1. We can use variables in CSS
        1. Why CSS variables?
            1. We don't need a pre-processor (if we are using Sass only for variables)
            2. They can be manipulated in JS
            3. We can edit them in Dev Tools
            4. We can use in Calc function
            5. They cascade and can be inherited
    2. They should be declared in a scope
        1. In a declaration block

            ```
            :root {

            }
            ```

            1. Similar to `html` selector but with higher specificity
            2. It is root of the document
                1. Whatever we declare here are available in the child elements
3. Code: _base.scss

```
/*
COLORS

Primary: #eb2f64
Primary light: #FF3366
Primary dark: #BA265D

Grey light 1: #faf9f9
Grey light 2: #f4f2f2
Grey light 3: #f0eeee
Grey light 4: #ccc

Grey dark 1: #333
Grey dark 2: #777
Grey dark 3: #999

*/

:root {
  --color-primary: #eb2f64;
  --color-primary-light: #ff3366;
  --color-primary-dark: #ba265d;

  --color-grey-light-1: #faf9f9;
  --color-grey-light-2: #f4f2f2;
  --color-grey-light-3: #f0eeee;
  --color-grey-light-4: #ccc;

  --color-grey-dark-1: #333;
  --color-grey-dark-2: #777;
  --color-grey-dark-3: #999;
}

* {
  margin: 0;
  padding: 0;
}

*,
*::before,
*::after {
  box-sizing: inherit;
}

html {
  box-sizing: border-box;
  font-size: 62.5%; // 1rem = 10px, 10px/16px = .625
}

body {
  font-family: "Open Sans", sans-serif;
  font-weight: 400;
  line-height: 1.6;
  color: var(--color-grey-dark-2);
```

```
    background-image: linear-gradient(
      to right bottom,
      var(--color-primary-light),
      var(--color-primary-dark)
    );
    background-size: cover;
    background-repeat: no-repeat;

    min-height: 100vh;
  }
```

Building the Overall Layout

What You Will Learn in this Lecture

1. How to think about the overall layout of an app
2. Use flexbox in a real-world project for the first time

Building the Header - Part 1

What You Will Learn in This Lecture

1. Why to use SVG icons vs. font icons
2. How to find, generate and use SVG sprites in HTML
3. How to change the color of an SVG icon in CSS
4. How to use more advanced flexbox alignment techniques, including `justify-content`, `align-items`, `align-self`, and `flex`

Implementation

1. SVGs
    1. Why not icon fonts?
        1. Hack to display icons which are like images using a font
        2. Fail more often
            1. If so, Browser display black square
        3. Screenreaders may not be able to read iconfonts
    2. SVG:
        1. Scalable Vector Graphics
            1. We write vector graphics using code
        2. icomoon.io
            1. IcoMoon App
                1. We can upload icon font and convert to SVGs
                2. Select icons from free library
                    1. Entypo+
                        1. Select
                        2. Download
                        3. Settings
                            1. Unselect PNG
        3. Sprite file (`symbol-defs.svg`)
            1. It has all the 10 images in it
                1. We can use only one global file
                    1. Only one HTTP request is sufficient
        4. demo.html
            1. Icon names
        5. We can use height and width to change the sizes of SVG
2. Code:

```
<nav class="user-nav">
    <div class="user-nav__icon-box">
        <svg class="user-nav__icon">
            <use xlink:href="img/sprite.svg#icon-bookmark"></use>
        </svg>
        <span class="user-nav__notification">7</span>
    </div>
    <div class="user-nav__icon-box">
        <svg class="user-nav__icon">
            <use xlink:href="img/sprite.svg#icon-chat"></use>
        </svg>
        <span class="user-nav__notification">13</span>
    </div>
    <div class="user-nav__user">
        <img src="img/user.jpg" alt="User photo" class="user-nav__user-photo">
        <span class="user-nave__user-name">Jonas</span>
    </div>
</nav>
```

Building the Header - Part 2

Building the Header - Part 3

1. Nested flex-boxes are allowed

Building the Navigation - Part 1

What you will Learn in this Lecture

1. How to use `scaleY` and multiple transition properties with different settings, to construct a creative hover effect

2. How and why to use the `currentColor` CSS variable
3. How to use some more advanced flexbox alignment techniques, including `flex-direction`, `justify-content`, and `align-items`

Building the Navigation - Part 2

1. `currentColor`: color of current element or parent element

Building the Hotel Overview - Part 1

What You will Learn in this Lecture

1. How to construct animation
2. How to use `margin: auto` with flexbox, and why it's so powerful
3. Continue to use flexbox properties for easy positioning and alignment

Building the Hotel Overview - Part 2

Building the Description Section - Part 1

What You will Learn in This Lecture

1. Continue to use flexbox, including `flex-wrap` to build a multi-column lis
2. How and why to use CSS masks with `mask-image` and `mask-size`

Building the Description Section - Part 2

1. It is not easy to use sprite image in CSS
2. Mask - Defines an area where we look through the element and see what's behind the element.
    1. We set background color of the element to a solid color
    2. We then use the icon as the mask
        1. We can see through the icon and the background becomes visible

Building the User Reviews Section

What You will Learn in This Lecture

1. Continue using and practicing flexbox
    1. figure - for review (they are good for pictures and some text)
2. css-tricks.com
    1. HTML elements

Building the CTA Section

What You will Learn in this Lecture

1. Yet another creative and modern hover efect
2. Transition does work for
    1. Background images
    2. Gradients

Writing Media Queries - Part 1

1. Flexbox makes it easy to make website responsive
2. CSS custom properties doesn't work in media query condition

Writing Media Queries - Part 2

Wrapping up the Trillo Project: Final Considerations

1. Final remarks
    1. caniuse.com
        1. flexbox
            1. Show all - which browsers support, and usage
2. Grid layout
    1. Good for overall layout
        1. Smaller details can be handled by flexbox

Challenges, Anyone?

1. Display some kind of user menu when hovering over the username in `.user-nav`
2. Display a message menu when hovering over the chat icon in `.user-nav` (maybe like facebook)
3. Display a box with search suggestions as soon as the user starts typing in the search field
4. Construct a caption for the `.gallery__item` with a nice hover effect
5. Make the page 100% responsive even for veiwport sizes below 500px, maybe even responsive images

Section 8: A Quick Introduction to CSS Grid Layouts

Section Intro

1. CSS Grids
    1. Makes building layouts easy

2. Project
3. Compare grid with flexbox

Why CSS Grid: A Whole New Mindset

A Whole New Mindset

1. CSS Grid
    1. CSS Grid layout is a brand new module that brings a two-dimensional grid system to CSS or the first time
    2. CSS Grid replaces float layouts, using less, and more readable and logical CSS and HTML
    3. CSS Grid works perfectly together with Flexbox, which is best to handle one-dimensional components and layouts
    4. CSS Grid completely changes the way that we envision and build two-dimensional layouts
        1. We don't need Bootstrap

A Whole New Mindset

CSS Grid Terminology

1. Grid container
    1. `display: grid`
        1. `display: grid-inline`
2. Column axis (y-direction)
    1. We cannot change direction
3. Row axis (x-direction)
    1. We cannot change direction
4. Grid items
    1. Direct children of grid container
5. Grid lines
    1. The lines that divide the rows and columns
    2. Automatically numbered
        1. 1 to (number of rows + 1)
        2. 1 to (number of columns + 1)
6. Gutter
    1. Space between rows or columns
        1. Row gutter
        2. Column gutter (can be different from row gutter)
7. Track
    1. Space between grid-lines (vertical or horizontal)
        1. Horizontal track: row
        2. Vertical track: column
8. Grid area
    1. Area between two vertical grid-lines and horizontal grid-lines
    2. Grid cell: If the area is between two adjacent grid-lines vertically and horizontally

CSS Grid Properties Overview

1. Container
    1. `grit-template`
        1. `grid-template-rows`
        2. `grid-template-columns`
        3. `grit-template-areas`
    2. `grig-gap`
        1. `grid-row-gap`
        2. `grid-column-gap`
    3. `justify-items`
    4. `align-items`
    5. `justify-content`
    6. `align-content`
    7. `grid-auto-rows`
    8. `grid-auto-columns`
    9. `grid-auto-flow`
2. Item
    1. `grid-area`
        1. `grid-row`
            1. `grid-row-start`
            2. `grid-row-end`
        2. `grid-column`
            1. `grid-column-start`
            2. `grid-column-end`
    2. `justify-self`
    3. `align-self`
    4. `order`

Quick Setup for This Section

1. New Browser:
    1. Firefox quantum
        1. Best Dev tools for CSS grids
            1. To visualize grid
2. Codepen
    1. Settings
        1. Oceanic Dark
        2. Source Code Pro
        3. 15px
3. Open Codepen in Firefox

Constructing Our First Grid

1. Code: html

```
<div class="container">
  <div class="item item--1">1: Orange</div>
  <div class="item item--2">2: Green</div>
  <div class="item item--3">3: Violet</div>
  <div class="item item--4">4: Pink</div>
  <div class="item item--5">5: Blue</div>
  <div class="item item--6">6: Brown</div>
</div>
```

2. Code: SCSS

```
.container {
  background-color: #eee;
  width: 1000px;
  margin: 30px auto;

  display: grid;
  grid-template-rows: 150px 150px;
  grid-template-columns: 150px 150px 150px;
  grid-row-gap: 30px;
  grid-column-gap: 30px;

  grid-gap: 30px;
}

.item {
  padding: 20px;
  font-size: 30px;
  font-family: sans-serif;
  color: white;

  &--1 {
    background-color: orangered;
  }

  &--2 {
    background-color: yellowgreen;
  }

  &--3 {
    background-color: blueviolet;
  }

  &--4 {
    background-color: palevioletred;
  }

  &--5 {
    background-color: royalblue;
  }

  &--6 {
    background-color: goldenrod;
  }
}
```

Getting Familiar with the fr Unit

1. `1fr` - occupies all the remaining space
    1. Fractional unit - the fraction of the available space (remaining space)
        1. `1fr` - all of the remaining space
        2. `.5fr` - 50% of the remaining space
    2. `grid-template-columns: 1fr 2fr 1fr`
        1. 1fraction of the available space
        2. 2fractions of the available space
        3. 1fraction of the available space
    3. `grid-template-columns: 50% 1fr 1fr;`
        1. First item: 500px (50% of the entire width)
            1. It doesn't consider gaps (`grid-gap`)

2. Code: SCSS

```
.container {
  background-color: #eee;
  width: 1000px;
  margin: 30px auto;

  height: 1000px;

  display: grid;
  grid-template-rows: repeat(2, 1fr); // fills up the container
  // grid-template-columns: repeat(3, 1fr);
  // grid-template-columns: 50% 1fr 1fr;
  grid-template-columns: repeat(3, 1fr);
  grid-row-gap: 30px;
  grid-column-gap: 30px;

  grid-gap: 30px;
}
```

Positioning Grid Items

```
&--5 {
  background-color: royalblue;
```

```
    // grid-row: 1 / 2;
    // grid-column: 3 / 4;
    grid-area: 1 / 3 / 2 / 4;
  }

  &--6 {
    background-color: goldenrod;
    grid-row: 1 / 2;
    grid-column: 2 / 3;
  }
```

Spanning Grid Items

1. Implicit grid:
    1. If a grid-cell occupies more space than available, the remaining cells are pushed to a new row called implicit grid
    2. If there are cells explicitly placed in a grid-row/grid-column, the other cells are moved accordingly
        1. If multiple cells are explicitly given overlapping locations, they will occupy the same space.
            1. We can set z-index to move one on top of the other
2. Code: SCSS

```
  &--1 {
    background-color: orangered;
    // grid-row-start: 2;
    // grid-row-end: 3;
    // grid-column-start: 2;
    // grid-column-end: 3;
    grid-row: 2 / 3;
    grid-column: 2 / 3;
    z-index: 10;
  }

  &--2 {
    background-color: yellowgreen;
    // grid-column: 2 / span 2;
    grid-column: 1 / -1;
  }

  &--3 {
    background-color: blueviolet;
    grid-row: 2 / 3;
    grid-column: 1 / 3;
  }

  &--4 {
    background-color: palevioletred;
  }

  &--5 {
    background-color: royalblue;
    // grid-row: 1 / 2;
    // grid-column: 3 / 4;
    grid-area: 1 / 3 / 2 / 4;
  }

  &--6 {
    background-color: goldenrod;
    grid-row: 1 / 2;
    grid-column: 2 / 3;
  }
```

Grid Challenge

1. Code: SCSS

```
.container {
  background-color: #eee;
  width: 500px;
  margin: 30px auto;

  height: 500px;

  display: grid;
  grid-template-rows: 10% repeat(3, 1fr) 10%;
  grid-template-columns: repeat(4, 1fr);
  grid-row-gap: 10px;
  grid-column-gap: 10px;

  grid-gap: 20px;
}

.item {
  padding: 10px;
  font-size: 20px;
  font-family: sans-serif;
  color: white;

  &--1 {
    background-color: orangered;
    grid-row: 1 / 2;
    grid-column: 1 / -1;
  }

  &--2 {
    background-color: orangered;
  }

  &--3 {
    background-color: orangered;
  }
```

```
    &--4 {
      background-color: orangered;
    }

    &--5 {
      background-color: orangered;
      grid-row: 2 / -2;
    }

    &--6 {
      background-color: orangered;
      grid-row: 3 / -2;
      grid-column: 1 / -2;
    }

    &--7 {
      background-color: orangered;
      grid-row: 5 / -1;
      grid-column: 1 / -1;
    }
  }
```

Grid Challenge: A Basic Solution

Naming Grid Lines

1. We can name grid-lines
    1. Convention: `<content-in-the-track>-<start|end>`
2. Code: Method 2

```
// METHOD 2: LINE NAMES
.container {
  background-color: #eee;
  width: 500px;
  margin: 30px auto;

  height: 500px;

  display: grid;
  grid-template-rows: [header-start] 10% [header-end box-start] 1fr [box-end main-start] 2fr [main-end footer-start] 10% [footer-end];
  grid-template-columns: repeat(4, [col-start] 1fr [col-end]); // Named-set: col-start-1, col-end-1, col-start-2, col-end-2, ...
  grid-row-gap: 10px;
  grid-column-gap: 10px;

  grid-gap: 20px;
}

.item {
  padding: 10px;
  font-size: 15px;
  font-family: sans-serif;
  color: white;

  &--1 {
    background-color: orangered;
    grid-row: header-start / header-end;
    grid-column: col-start 1 / col-end -1;
  }

  &--2 {
    background-color: orangered;
  }

  &--3 {
    background-color: orangered;
  }

  &--4 {
    background-color: orangered;
  }

  &--5 {
    background-color: orangered;
    grid-row: box-start / main-end;
  }

  &--6 {
    background-color: orangered;
    grid-row: main-start / main-end;
    grid-column: col-start 1 / col-end -2;
  }

  &--7 {
    background-color: orangered;
    grid-row: footer-start / footer-end;
    grid-column: col-start 1 / col-end -1;
  }
}
```

Naming Grid Areas

Implicit Grids vs. Explicit Grids

1. Code: html

```
<div class="container">
  <div class="item item--1">Modern</div>
```

```
  <div class="item item--2">CSS</div>
  <div class="item item--3">with</div>
  <div class="item item--4">Flexbox</div>
  <div class="item item--5">and</div>
  <div class="item item--6">Grid</div>
  <div class="item item--7">is</div>
  <div class="item item--8">great</div>
</div>
```

2. Code: SCSS

```
.container {
  width: 1000px;
  margin: 30px auto;
  background-color: #ddd;

  display: grid;
  grid-template-rows: repeat(2, 150px);
  grid-template-columns: repeat(2, 1fr);
  grid-gap: 30px;

  grid-auto-rows: 80px; // automatically added rows have 80px height
  grid-auto-flow: column; // remaining items are added as columns. row - default. All items fill columns first instead of rows
  grid-auto-columns: .5fr; // width is set to 1/2 the width of regular cells

  .item {
    padding: 20px;
    color: white;
    font-family: sans-serif;
    font-size: 30px;
    background-color: orangered;
  }
}
```

    1. 2 x 2 we defined is explicit grid
    2. Rest of the grid is implicit grid
        1. Tracks get automatically added
    3. Useful if we don't know how many items will be received from server.

Aligning Grid Items

1. `align-items` by default is set to `stretch`
    1. `align-items: center` - centers the items vertically
2. `justify-items` by default is set to `stretch` (since grid is 2D)
    1. `justify-items: center` - centers the items horizontally
3. `align-self`
4. `justify-self`

\

Aligning Tracks

1. `justify-content` - aligns items wrt container
    1. Similar to flex-box

```
justify-content: space-evenly; // center, start, end, space-between, space-around, space-evenly
```

2. `align-content` - aligns items wrt container vertically

```
align-content: center; // center, start, end, space-between, space-around, space-evenly
```

3. Dense grid with no holes

```
grid-auto-flow: row dense;
```

    1. Useful for images

4. Code: SCSS

```
.container {
  width: 1000px;
  margin: 30px auto;
  background-color: #ddd;

  display: grid;
  // grid-template-rows: repeat(2, 150px);
  // grid-template-columns: repeat(2, 1fr);
  grid-gap: 30px;

  grid-auto-rows: 80px; // automatically added rows have 80px height
  grid-auto-flow: row dense;
  grid-auto-columns: .5fr;

  // Align grid items to grid areas
  // align-items: center;
  // justify-items: center;

  // Align grid tracks to grid container
  grid-template-rows: repeat(2, 100px);
  grid-template-columns: repeat(2, 200px);

  height: 1000px;

  justify-content: space-evenly; // center, start, end, space-between, space-around, space-evenly
  align-content: center; // center, start, end, space-between, space-around, space-evenly

  .item {
    padding: 10px;
```

```
          color: white;
          font-family: sans-serif;
          font-size: 30px;
          background-color: orangered;

          &--4 {
            background-color: crimson;
            grid-row: 2 / span 3;
            align-self: end;
            justify-self: end;
          }

          &--6 {
            background-color: lightcoral;
            grid-row: 2 / span 2;
          }

          &--7 {
            background-color: palevioletred;
            grid-column: 1 / -1;
          }
        }
      }
```

Using min-content, max-content, and the minmax() function

1. `max-content` - maximum size of the content without overflowing
2. `minmax(150px, min-content)` - size should be at least 150px but can gro upto min-content if required.
    1. Between 150px and the min content height
    2. `minmax(200px, 1fr)`
        1. It is `1fr` but will never less than the minimum content width.

Responsive Layouts with auto-fit and auto-fill

1. No media query required
2. Code: SCSS

```
// Using auto-fill and auto-fit
display: grid;
grid-template-rows: repeat(2, minmax(150px, min-content));
grid-template-columns: repeat(auto-fill, 100px); // 10 column tracks are created because container is 1000px. 1000/100 = 10
grid-template-columns: repeat(auto-fit, 100px); // same as auto-fill but collapses extra tracks to have a width of 0.

// grid-template-columns: repeat(auto-fit, 100px); // same as auto-fill but collapses extra tracks to have a width of 0.
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr)); // minimum 100px but fills the container. Overlowing grid item goes to the next
```

    1. Useful for responsive layouts without writing media-query

Section 9: Nexter Project - Master CSS Grid Layouts!

Project Overview and Setup

1. Nexter project
2. Live server in firefox

```
"devserver": "live-server --browser=firefox",
```

Building the Overall Layout - Part 1

What You will Learn in This Lecture

1. How to build a complex and modern layout using advanced CSS Grid techniques
2. How to choose different row and column track sizes for different types of content

Building the Overall Layout - Part 2

1. Predefined 8/10/12 column grid can be used

Building the Features Section - Part 1

What You will Learn in This Lecture

1. How and why to construct grids inside of grids
2. How to construct a responsive component without media queries
3. How to build a small component using CSS grid

Implementation

1. Subgrid is not available now
    1. Aligning columns with parent grid columns is not possible
        1. Manual creation of grid is required

Building the Features Section - Part 2

What You will Learn in This Lecture

1. How to deal with overlapping grid items

2. Why images are special and behave differently than other grid items
3. How to decide if flexbox is a better tool in certain situations

Implementation

1. Flexbox is good for 1d positioning

Building the Story Section - Part 1

1. Images have aspect ratio
    1. It tries to keep its aspect ratio (doesn't fill at times)

Building the Story Section - Part 2

What You Will Learn in this Lecture

1. How to build a rather complex component using a mix of CSS Grid properties, overlapping and flexbox

Building the Homes Section - Part 1

Building the Homes Section - Part 2

2. Flexbox is good for small alignments

Building the Gallery - Part 1

What You will Learn in This Lecture

1. How to construct a complex grid-looking gallery
2. Using `object-fit` together with images for grid items

Implementation

1. Consider the smaller unit
    1. Try to divide based on that unit
        1. The smaller the cells, the more variaty we can produce

Building the Gallery - Part 2

Building the Footer

What You Will Learn in This Lecture

1. Apply the concepts you already learned

Building the Sidebar

Building the Header - Part 1

What You Will Learn in This Lecture

1. How to manage vertical spacing in a responsive layout using CSS Grid techniques
2. How to use `::before` and `::after` as grid items

Building the Header - Part 2

1. If rows are skipped in `grid-template-rows`, the cell will be sized based on the content (`min-content`)

Building the Realtors Section

Writing Media Queries - Part 1

Writing Media Queries - Part 2

Browser Support to CSS Grid

1. CSS Grid support:
    1. caniuse.com
        1. https://caniuse.com/?search=css%20grid - 96.63% support
            1. Depends on who the users are
2. Progressive enhancement (duplicate work, messing with old code)
    1. We first build a website with older CSS methods pretending that the new features do not exist
    2. We then build a layer on top of that with modern CSS features

Progressive Enhancement

1. Grid containers and grid items ignore certain things
    1. `float`
    2. `display: inline-block`
    3. `display: table-cell`

    4. Vertical align

2. Code: Progressive enhancement

```
.features {
  // background-color: $color-grey-light-2;
  grid-column: center-start / center-end;

  margin: 15rem 0;

  @supports (display: grid) {
    display: grid;
    // grid-template-columns: repeat(3, 1fr);
    grid-template-columns: repeat(auto-fit, minmax(25rem, 1fr));
    // rows are placed in implict grid
    grid-gap: 6rem;
    align-items: start;
  }
}

.feature {
  float: left;
  width: 33.3333%;
  margin-bottom: 6rem;

  @supports (display: grid) {
    display: grid;
    grid-template-columns: min-content 1fr; // Defining only columns is common
    grid-row-gap: 1.5rem;
    grid-column-gap: 2.5rem;

    width: auto;
    margin-bottom: 0;
  }
  // ...
}
```

Wrapping up the Nexter Project: Final Considerations

1. Running build script
2. Autoprefixer turns of prefixes for CSS grid (not required)
3. `grid-template`

```
grid-template: repeat(7, 5vw) / repeat(8, 1fr);
```

4. Good to have features:
    1. Subgrid snapping to parent grid columns
    2. Ability to change individual gaps
    3. Selector for mth row and nth column
        1. Even rows
        2. Odd rows
        3. ...

Section 10: That's it, Everyone!

See You Next Time, CSS Master!

1. Come up with own ideas
2. Own layouts
3. Own designs
4. Build using the tools given in the course
5. Share work
6. Keep up with latest news
    1. jonas.io - newsletter
7. Never stop learning

My Other Courses + Updates

1. `@jonasschmedtman`
2. Sign up or mailing list
    1. Discounts for future courses
3. YouTube channel